

Uso básico de GDB

GDB permite detener un programa mientras se está ejecutando, avanzar línea por línea y revisar el valor de sus variables. Es especialmente útil cuando el programa se cae, entra en un ciclo inesperado o entrega un resultado distinto al esperado.

Esta guía muestra los comandos básicos para comenzar. No es necesario memorizar todo: lo importante es entender el flujo general y saber qué información buscar.

1 Compilar para usar GDB

Compila el programa con la opción `-g`. Esta opción agrega la información que GDB necesita para mostrar nombres de funciones, archivos y números de línea. Durante la depuración también conviene desactivar optimizaciones con `-O0`:

```
gcc -Wall -Wextra -Wpedantic -g -O0 -o programa programa.c
```

Al igual que lo explicado en la guía de valgrind, en los archivos `Makefile` ya estarán configuradas las flags necesarias.

Después abre el ejecutable con:

```
gdb ./programa
```

Cuando veas el prompt (`gdb`), significa que GDB está esperando un comando.

2 Detenerse en una línea

Un *breakpoint* es un punto donde GDB debe pausar el programa. Puedes detenerte al entrar a una función:

```
break main
```

También puedes indicar una línea concreta:

```
break programa.c:13
```

Luego inicia el programa con:

```
run
```

La siguiente imagen muestra una sesión en la que el programa se detiene al comenzar `main`.

```
estudiante@nixos-curso-c:~/Trabajo$ gcc -Wall -Wextra -Wpedantic -g -O0 -o programa programa.c
estudiante@nixos-curso-c:~/Trabajo$ gdb ./programa
GNU gdb (GDB)
Reading symbols from ./programa...
(gdb) break main
Breakpoint 1 at 0x118c: file programa.c, line 13.
(gdb) run
Starting program: /home/estudiante/Trabajo/programa

Breakpoint 1, main () at programa.c:13
13      int total = sumar(datos, 4);
(gdb)
```

En la última parte de la salida aparecen tres datos útiles:

- el breakpoint donde se detuvo el programa;
- el archivo y número de línea, por ejemplo `programa.c:13`;
- la instrucción que está a punto de ejecutarse.

Como atajo, el comando `start` inicia el programa y se detiene automáticamente al comienzo de `main`.

3 Avanzar por el programa

Una vez que el programa está detenido, estos son los comandos más usados:

- `next`: ejecuta la línea actual. Si llama a una función, no entra en ella. Por lo que, no verás la ejecución de esa función, veras que sigues en la siguiente línea después del llamado.
- `step`: ejecuta la línea actual y entra en la función llamada.
- `continue`: continúa hasta el siguiente breakpoint o hasta que el programa termine.
- `finish`: continúa hasta que termine la función actual.

Puedes repetir el último comando presionando Enter en una línea vacía. Esto es cómodo para avanzar varias veces con `next` o `step`.

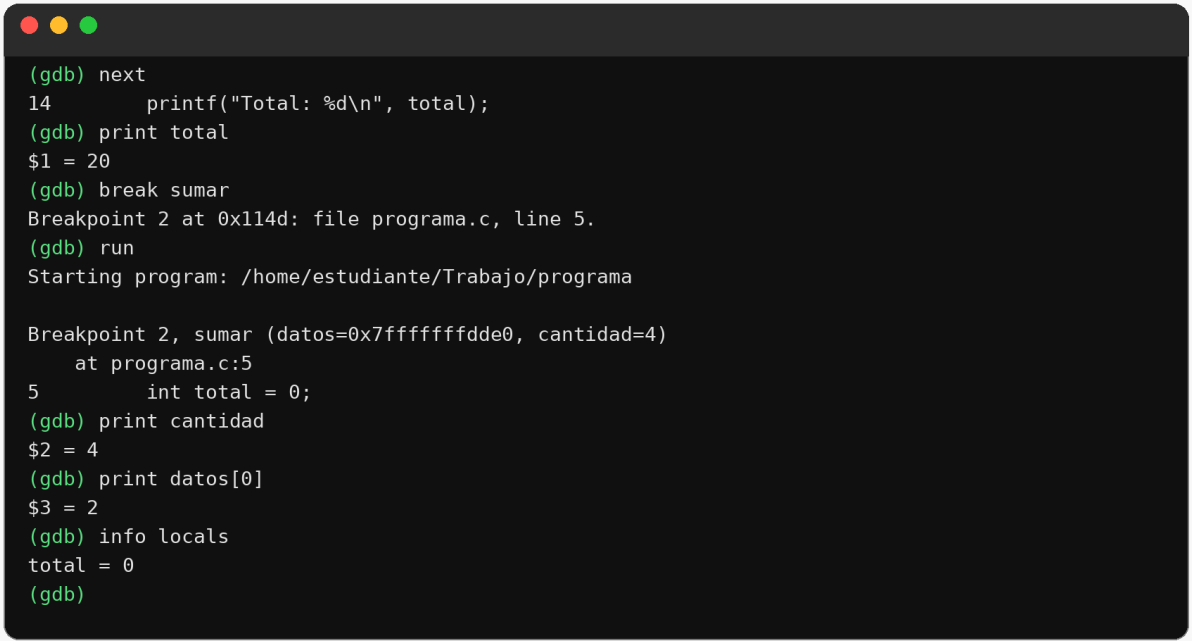
4 Revisar variables

El comando `print` muestra el valor de una expresión:

```
print total
print i
print datos[0]
print *puntero
```

También puedes usar:

- `info locals`: muestra las variables locales de la función actual.
- `info args`: muestra los argumentos recibidos por la función.
- `display variable`: muestra automáticamente una variable cada vez que el programa se detiene.
- `undisplay`: deja de mostrar esas expresiones automáticamente.

A screenshot of a GDB terminal window. The window has a dark background with light-colored text. At the top, there are three colored circles (red, yellow, green) representing window controls. The text in the terminal shows a sequence of GDB commands and their outputs. The commands include 'next', 'print total', 'break sumar', 'run', 'print cantidad', 'print datos[0]', and 'info locals'. The outputs show memory addresses like '\$1 = 20', '\$2 = 4', and '\$3 = 2', as well as program execution details like 'Starting program: /home/estudiante/Trabajo/programa' and 'Breakpoint 2, sumar (datos=0x7fffffffddde0, cantidad=4) at programa.c:5'.

```
(gdb) next
14      printf("Total: %d\n", total);
(gdb) print total
$1 = 20
(gdb) break sumar
Breakpoint 2 at 0x114d: file programa.c, line 5.
(gdb) run
Starting program: /home/estudiante/Trabajo/programa

Breakpoint 2, sumar (datos=0x7fffffffddde0, cantidad=4)
at programa.c:5
5      int total = 0;
(gdb) print cantidad
$2 = 4
(gdb) print datos[0]
$3 = 2
(gdb) info locals
total = 0
(gdb)
```

Los valores impresos por GDB suelen aparecer como \$1, \$2, etc. Esos números son nombres temporales que GDB asigna a los resultados. No forman parte de tu programa.

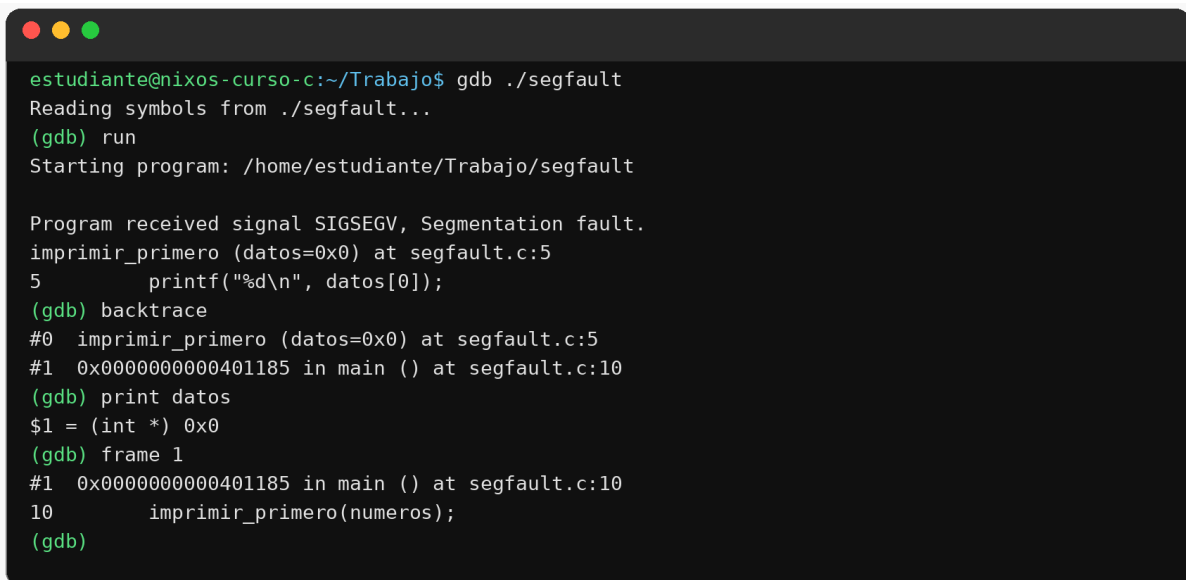
5 Cuando el programa se cae

Si el programa produce un *segmentation fault*, ejecútalo dentro de GDB. Cuando ocurra el error, GDB se detendrá exactamente en el punto donde el sistema detectó el acceso inválido.

El comando más importante en este caso es:

```
backtrace
```

También puede escribirse como `bt`. Muestra la cadena de funciones que llevó hasta el error.



```
estudiante@nixos-curso-c:~/Trabajo$ gdb ./segfault
Reading symbols from ./segfault...
(gdb) run
Starting program: /home/estudiante/Trabajo/segfault

Program received signal SIGSEGV, Segmentation fault.
imprimir_primer (datos=0x0) at segfault.c:5
5      printf("%d\n", datos[0]);
(gdb) backtrace
#0  imprimir_primer (datos=0x0) at segfault.c:5
#1  0x0000000000401185 in main () at segfault.c:10
(gdb) print datos
$1 = (int *) 0x0
(gdb) frame 1
#1  0x0000000000401185 in main () at segfault.c:10
10      imprimir_primer(numeros);
(gdb)
```

En el ejemplo anterior:

1. Program received signal SIGSEGV indica que el programa intentó usar memoria inválida.
2. segfault.c:5 es la línea donde el programa se cayó.
3. backtrace muestra que main llamó a imprimir_primer.
4. print datos revela que el puntero vale 0x0, es decir, es NULL.

6 Detener un programa que no termina

Si un programa queda atrapado en un ciclo, ejecútalo con `run` y presiona `Ctrl+C`. GDB interrumpirá la ejecución sin cerrar la sesión.

```
(gdb) run
Starting program: /home/estudiante/Trabajo/ciclo
^C
Program received signal SIGINT, Interrupt.
main () at ciclo.c:8
8      while (i < limite) {
(gdb) print i
$1 = 3
(gdb) print limite
$2 = 10
(gdb) next
9          total += i;
(gdb)
```

Desde ese punto puedes revisar variables y avanzar con **next**. Esto suele ayudar a encontrar una condición que nunca cambia o una variable que no se actualiza como esperabas.

7 Programas con argumentos

Puedes entregar los argumentos al iniciar GDB:

```
gdb --args ./programa archivo.txt 20
```

O configurarlos desde una sesión abierta:

```
set args archivo.txt 20
run
```

Para revisar los argumentos actuales:

```
show args
```

8 Breakpoints útiles

Además de **break**, puedes usar:

- **info breakpoints**: muestra los breakpoints existentes.
- **delete 1**: elimina el breakpoint número 1.
- **disable 1**: lo desactiva sin eliminarlo.
- **enable 1**: vuelve a activarlo.
- **clear funcion**: elimina el breakpoint asociado a esa función.

También puedes pausar solo cuando se cumpla una condición:

```
break programa.c:20 if i == 50
```

Esto es útil para ciclos largos, porque evita avanzar manualmente hasta la iteración que te interesa.

9 Resumen de comandos

Comando	Qué hace
<code>run</code>	Inicia el programa.
<code>start</code>	Inicia y se detiene al comienzo de <code>main</code> .
<code>break lugar</code>	Crea un breakpoint.
<code>next</code>	Avanza sin entrar en funciones.
<code>step</code>	Avanza entrando en funciones.
<code>continue</code>	Continúa hasta la próxima pausa.
<code>print expresion</code>	Muestra el valor de una expresión.
<code>info locals</code>	Muestra variables locales.
<code>backtrace</code>	Muestra la cadena de llamadas.
<code>list</code>	Muestra líneas del código fuente.
<code>quit</code>	Cierra GDB.

10 Flujo recomendado

Cuando no sepas por qué un programa falla, prueba este orden:

1. Compila tu código.
2. Abre el ejecutable con GDB.
3. Usa `start` o agrega un breakpoint cerca del problema.
4. Avanza con `next` y entra con `step` solo cuando sea necesario.
5. Revisa variables con `print`, `info locals` e `info args`.
6. Si el programa se cae, usa `backtrace`.
7. Si no termina, interrúmpelo con `Ctrl+C`.

GDB no reemplaza leer el código ni pensar en el problema, pero permite comprobar paso a paso qué está haciendo realmente el programa.

Sabemos que GDB es una herramienta que al inicio parece difícil de usar, pero con un poco de práctica se hace bastante intuitiva y útil.